VITE & GOURMAND

Gestion de projet

Application Web Traiteur Événementiel

Projet réalisé dans le cadre du titre professionnel
Développeur Web et Web Mobile

Version : 1.0

Date : Mars 2026

Projet : Vite & Gourmand

Sommaire

01 Présentation du projet

02 Méthodologie

03 Organisation du projet

04 Outils utilisés

05 Organisation des tâches

06 Cycle de développement

07 Difficultés rencontrées

08 Solutions mises en place

01 — Présentation du projet

1.1 Objectif

Vite & Gourmand est une application web complète pour un service de traiteur événementiel basé à Bordeaux. L'objectif est de digitaliser l'ensemble du parcours client : de la consultation des menus à la commande en ligne, en passant par la demande de devis personnalisé pour des événements.

1.2 Contexte

Le projet a été développé dans le cadre de l'ECF (Évaluation en Cours de Formation) du titre professionnel **Développeur Web et Web Mobile (DWWM)**. Il couvre l'ensemble des compétences attendues :

- Développement frontend (React)
- Développement backend (API REST Node.js/Express)
- Base de données relationnelle (PostgreSQL)
- Authentification et sécurité (JWT, bcrypt)
- Déploiement avec Docker
- Conformité RGPD

1.3 Périmètre fonctionnel

Module	Description
Authentification	Inscription, connexion, réinitialisation mot de passe, gestion JWT
Catalogue menus	Consultation, filtrage, détail avec composition et avis
Commande en ligne	Formulaire de commande avec calcul de prix en temps réel
Module devis	Demande de devis multi-étapes pour événements
Suivi commandes	Timeline de statut, historique, annulation
Système d'avis	Publication, modération, affichage public
Espace employé	Gestion commandes, menus, plats, devis, avis
Administration	Gestion utilisateurs, pages légales, analytiques
Pages vitrine	Accueil, prestations, boissons, contact, pages légales

Module	Description
RGPD	Consentement, export de données, suppression de compte

02 — Méthodologie

2.1 Approche Agile simplifiée

Le développement a suivi une méthodologie **Agile simplifiée**, adaptée à un projet individuel :

- **Découpage en fonctionnalités** : chaque module (authentification, menus, commandes, devis, etc.) a été traité comme une fonctionnalité indépendante
- **Itérations courtes** : développement par sprints de fonctionnalités, avec livraison incrémentale
- **Tests continus** : validation de chaque fonctionnalité avant de passer à la suivante
- **Adaptation** : le périmètre a évolué au fil du développement (ajout du module devis, des formules boissons, etc.)

2.2 Principes appliqués

Principe	Application
Livraison incrémentale	Chaque module est fonctionnel indépendamment avant de passer au suivant
Priorisation	Les fonctionnalités critiques (auth, menus, commandes) ont été développées en premier
Refactoring continu	Le code a été restructuré en architecture OOP (Controller → Service → Repository) après la phase initiale
Documentation au fil de l'eau	Les choix techniques sont documentés à mesure

03 — Organisation du projet

3.1 Structure du dépôt

```
vite-gourmand/
├── backend/                    # API Node.js / Express
│   ├── src/
│   │   ├── controllers/        # Contrôleurs HTTP
│   │   ├── services/           # Logique métier
│   │   ├── repositories/       # Accès données (SQL)
│   │   ├── middleware/         # Auth, validation
│   │   ├── routes/             # Définition des routes
│   │   ├── utils/              # Helpers (validators, logger)
│   │   ├── scripts/            # Scripts de seed
│   │   └── index.js            # Point d'entrée
│   ├── Dockerfile              # Image de développement
│   ├── Dockerfile.prod         # Image de production
│   └── package.json
├── frontend/                   # Application React / Vite
│   ├── src/
│   │   ├── pages/              # Pages de l'application
│   │   ├── components/         # Composants réutilisables
│   │   ├── context/            # Contexte React (auth)
│   │   ├── styles/             # CSS design system
│   │   └── App.jsx             # Router principal
│   ├── Dockerfile              # Image de développement
│   ├── Dockerfile.prod         # Image de production (multi-stage)
│   └── package.json
├── database/                   # Scripts SQL
│   ├── schema.sql              # Schéma principal
│   ├── seed.sql                # Données de démonstration
│   ├── quotes_migration.sql        # Migration devis
│   ├── payments_migration.sql      # Migration paiements
│   └── quotes-deposit-request.sql  # Migration acompte
├── docs/                       # Documentation projet
├── docker-compose.yml          # Orchestration Docker
├── .env.example                # Variables d'environnement
└── README.md
```

3.2 Gestion Git

Le projet utilise **Git** comme système de contrôle de version, hébergé sur **GitHub**.

3.3 Stratégie de branches

Branche	Rôle
`main`	Branche de production, code stable et livrable
`develop`	Branche de développement, intégration des fonctionnalités
`feature/*`	Branches de fonctionnalité, créées depuis `develop`

3.4 Workflow Git

```
feature/auth ──┐
               ▼
feature/menus ──→ develop ──→ main
               ▲
feature/orders ──┘
```

1. Créer une branche `feature/nom-fonctionnalite` depuis `develop`
2. Développer la fonctionnalité avec des commits atomiques
3. Tester localement
4. Merger dans `develop`
5. Après validation de l'ensemble, merger `develop` dans `main`

3.5 Conventions de commit

Les messages de commit suivent un format descriptif :

```
<type>: <description>

Exemples :
- feat: add user authentication with JWT
- fix: correct order pricing calculation
- refactor: restructure to OOP architecture
- docs: add deployment documentation
```

04 — Outils utilisés

4.1 Développement

Outil	Version	Usage
Node.js	20 LTS	Runtime JavaScript backend
npm	10+	Gestionnaire de paquets
React	18+	Framework frontend (SPA)
Vite	5+	Bundler et serveur de développement frontend
Express.js	4.21	Framework HTTP backend
PostgreSQL	16	Base de données relationnelle principale
MongoDB	7	Base de données NoSQL (analytiques)

4.2 Infrastructure et déploiement

Outil	Usage
Docker	Conteneurisation de tous les services
Docker Compose	Orchestration multi-conteneurs (5 services)
MailHog	Interception et test des emails en développement
Nginx	Reverse proxy et serveur de fichiers statiques (production)

4.3 Sécurité

Outil / Bibliothèque	Usage
jsonwebtoken	Génération et vérification des tokens JWT
bcrypt	Hachage sécurisé des mots de passe (12 salt rounds)
helmet	Headers HTTP sécurisés
express-rate-limit	Limitation de requêtes (anti-brute force)

Outil / Bibliothèque	Usage
express-validator	Validation et assainissement des entrées
cors	Contrôle des origines autorisées

4.4 Qualité et documentation

Outil	Usage
Jest	Framework de tests unitaires
Pino	Logger structuré (JSON en production)
react-helmet-async	SEO — balises meta et titres par page
Draw.io	Diagrammes UML et MERISE
GitHub	Hébergement du code source et collaboration
Trello	Gestion de projet Kanban — suivi des tâches et sprints

4.5 Bibliothèques métier

Bibliothèque	Usage
pg	Client PostgreSQL pour Node.js
mongoose	ODM MongoDB
nodemailer	Envoi d'emails SMTP
uuid	Génération d'identifiants uniques
dotenv	Chargement des variables d'environnement

05 — Organisation des tâches

5.1 Gestion des tâches avec Trello

Le suivi des tâches du projet est réalisé via un **tableau Trello Kanban**, organisé en 5 colonnes reflétant le cycle de vie de chaque tâche :

Colonne	Rôle
Backlog	Fonctionnalités identifiées, non encore planifiées
À faire	Tâches priorisées pour le sprint en cours
En cours	Tâches en développement actif
En test	Tâches développées, en phase de validation et tests
Terminé	Tâches validées, testées et mergées

Chaque carte Trello contient :

- **Un titre** décrivant la fonctionnalité ou la tâche
- **Un label de couleur** indiquant la catégorie (Frontend, Backend, Base de données, DevOps, Documentation, Sécurité)
- **Une description** résumant le périmètre de la tâche
- **Une checklist** détaillant les sous-tâches à réaliser

Cette organisation permet de visualiser l'avancement global du projet en un coup d'œil et de prioriser les tâches selon les besoins du sprint.

5.2 Accès au tableau Trello

Le tableau Trello du projet est accessible en ligne :

https://trello.com/b/QMxXKb6W

5.3 Backlog fonctionnel

Le développement a suivi l'ordre de priorité suivant :

#	Fonctionnalité	Priorité	Statut
1	**Authentification** (inscription, connexion, JWT, rôles)	Critique	Terminé

#	Fonctionnalité	Priorité	Statut
2	**Gestion des menus** (CRUD, images, composition)	Critique	Terminé
3	**Gestion des plats** (CRUD, association aux menus)	Critique	Terminé
4	**Système de commande** (formulaire, calcul prix, stock)	Critique	Terminé
5	**Suivi des commandes** (statuts, timeline, historique)	Haute	Terminé
6	**Système d'avis** (publication, modération)	Haute	Terminé
7	**Espace employé** (dashboard, gestion commandes/menus)	Haute	Terminé
8	**Panel administration** (utilisateurs, pages légales)	Haute	Terminé
9	**Module devis** (demande, suivi, acompte, conversion)	Moyenne	Terminé
10	**Pages vitrine** (accueil, prestations, boissons, contact)	Moyenne	Terminé
11	**Réinitialisation mot de passe** (token, email, formulaire)	Moyenne	Terminé
12	**Changement de mot de passe** (depuis le profil)	Moyenne	Terminé
13	**RGPD** (consentement, export, suppression)	Haute	Terminé
14	**SEO** (balises meta, titres par page)	Basse	Terminé
15	**Analytiques** (statistiques admin, MongoDB)	Basse	Terminé
16	**Déploiement Docker** (dev + prod)	Haute	Terminé
17	**Pages légales** (CRUD admin, affichage public)	Moyenne	Terminé
18	**Contact** (formulaire, envoi email)	Basse	Terminé

5.4 Découpage d'une fonctionnalité type

Exemple pour le **système de commande** :

1. Backend

- Modèle de données (table `orders`, `order_status_history`)
- Repository (requêtes SQL : create, findById, list, updateStatus)
- Service (logique métier : validation, calcul prix, gestion stock, transaction)
- Contrôleur (endpoints HTTP : POST, GET, PUT, DELETE)
- Routes et middleware (validation, authentification)

- Tests unitaires

2. Frontend

- Page de commande (`OrderPage.jsx`) avec formulaire
- Calcul de prix en temps réel (preview)
- Page de détail commande (`OrderDetail.jsx`)
- Liste des commandes dans le dashboard (`UserDashboard.jsx`)
- Timeline de statut
- Gestion des erreurs et messages

3. Intégration

- Tests end-to-end du flux complet
- Vérification des emails de confirmation
- Validation de la gestion du stock

06 — Cycle de développement

6.1 Développement

1. **Analyse** : étude du besoin, identification des entités et relations
2. **Base de données** : création des tables, contraintes, index
3. **Backend** : développement en couches (Repository → Service → Controller → Route)
4. **Frontend** : création des pages et composants React
5. **Intégration** : connexion frontend/backend via l'API REST

6.2 Tests

Type de test	Outil	Couverture
Tests unitaires	Jest	Services backend (PricingService, AuthService)
Tests d'intégration	Jest + supertest	Endpoints API
Tests manuels	Navigateur + MailHog	Parcours utilisateur complets
Validation RGPD	Manuel	Consentement, export, anonymisation

Exemples de tests implémentés :

- Calcul du prix de commande (prix menu, livraison, remise groupe)
- Authentification (inscription, connexion, token invalide)
- Gestion des rôles (accès refusé pour un utilisateur non autorisé)
- Validation des entrées (email invalide, mot de passe trop faible)

6.3 Intégration

- **Docker Compose** assure que tous les services démarrent dans le bon ordre grâce aux **healthchecks** :
  - PostgreSQL démarre en premier → healthcheck `pg_isready`
  - MongoDB démarre en parallèle → healthcheck `mongosh ping`
  - L'API attend que PostgreSQL et MongoDB soient healthy
  - Le frontend attend que l'API soit healthy
- Les scripts SQL sont exécutés automatiquement au premier démarrage
- Les données de démonstration sont chargées via les scripts de seed

6.4 Déploiement

Le projet propose 3 stratégies de déploiement documentées :

Stratégie	Usage
Docker Compose (dev)	Environnement de développement local complet
VPS + Nginx + PM2	Déploiement sur serveur dédié
PaaS (Render, Vercel)	Déploiement cloud managé

Images Docker de production :

- Frontend : build multi-stage (Node pour le build, Nginx Alpine pour servir les fichiers statiques — ~30 Mo)
- Backend : image Node Alpine avec `npm ci --omit=dev` (dépendances de production uniquement)

07 — Difficultés rencontrées

7.1 Gestion transactionnelle des commandes

Problème : La création d'une commande implique plusieurs opérations interdépendantes : vérification du menu, décrémentation du stock, calcul du prix, insertion de la commande et de l'historique de statut. Si l'une de ces opérations échoue, les données doivent rester cohérentes.

Risque : Sans transaction, un crash entre la décrémentation du stock et l'insertion de la commande pourrait entraîner une perte de stock sans commande associée.

7.2 Sécurité de l'authentification

Problème : L'authentification est un vecteur d'attaque majeur. Les risques incluent : attaques par force brute, vol de token JWT, timing attacks sur la réinitialisation de mot de passe, et injection dans les formulaires.

Risque : Un système d'authentification mal conçu peut compromettre l'ensemble des données utilisateur.

7.3 Réinitialisation du mot de passe

Problème : Le flux de réinitialisation de mot de passe doit être sécurisé contre les attaques par énumération (un attaquant ne doit pas pouvoir déterminer si un email est inscrit ou non) et les tokens doivent être suffisamment aléatoires et avoir une durée de vie limitée.

7.4 Déploiement Docker avec volumes nommés

Problème : Les volumes Docker nommés (`api_node_modules`, `frontend_node_modules`) isolent les `node_modules` du système hôte. Cependant, après l'ajout d'une nouvelle dépendance dans `package.json`, le conteneur utilise toujours l'ancien volume et ne voit pas la nouvelle dépendance.

7.5 Calcul du prix de livraison

Problème : Le prix de livraison dépend de la distance entre Bordeaux et l'adresse du client. Le calcul doit être effectué côté frontend (pour le preview) et vérifié côté backend (pour la facturation).

7.6 Concurrence d'accès au stock

Problème : Si deux clients commandent le dernier menu disponible en même temps, le stock pourrait devenir négatif sans gestion de concurrence.

7.7 Architecture initiale monolithique

Problème : Le code backend initial mélangeait logique métier, accès aux données et gestion HTTP dans les mêmes fichiers, rendant le code difficile à maintenir et à tester.

08 — Solutions mises en place

8.1 Transactions SQL (commandes)

La création de commande utilise une **transaction PostgreSQL** avec `BEGIN`, `COMMIT` et `ROLLBACK` :

```
BEGIN
  → Vérifier que le menu est actif
  → Vérifier que nb_persons ≥ min_persons
  → Décrémenter le stock (UPDATE atomique avec WHERE stock > 0)
  → Calculer les prix (PricingService)
  → Insérer la commande
  → Insérer l'historique de statut
COMMIT
```

Si une étape échoue, `ROLLBACK` annule toutes les modifications. La décrémentation du stock est **atomique** grâce à une clause SQL `WHERE stock > 0 RETURNING stock`, qui empêche les stocks négatifs même en cas d'accès concurrent.

8.2 JWT et sécurité d'authentification

Mesure	Détail
Tokens JWT	Access token de 15 minutes + refresh token de 7 jours
bcrypt	Hachage des mots de passe avec 12 salt rounds
Rate limiting	200 req/15 min (général), 20 req/15 min (endpoints auth)

Mesure	Détail
Helmet.js	Headers de sécurité HTTP (CSP, HSTS, X-Frame-Options)
express-validator	Validation et assainissement systématique de toutes les entrées
CORS	Origines restreintes aux domaines autorisés
Politique de mot de passe	Min. 10 caractères, majuscule, minuscule, chiffre, caractère spécial

8.3 Réinitialisation sécurisée du mot de passe

Mesure	Détail
Token cryptographique	`crypto.randomBytes(32)` — 64 caractères hexadécimaux
Expiration	Token valide 1 heure uniquement
Usage unique	Token supprimé après utilisation
Anti-timing attack	Réponse identique que l'email existe ou non (HTTP 200)
Rate limiting	20 req/15 min sur les endpoints d'authentification

8.4 Gestion des volumes Docker

Solution documentée et appliquée :

```
# Après ajout d'une dépendance
docker exec vg_api npm install && docker restart vg_api
docker exec vg_frontend npm install && docker restart vg_frontend
```

Les volumes nommés persistent les `node_modules` dans Docker, isolés du système hôte. La commande `npm install` à l'intérieur du conteneur met à jour le volume, et le redémarrage recharge l'application avec la nouvelle dépendance.

8.5 Calcul du prix de livraison

- Livraison **gratuite** pour Bordeaux (détection par le nom de ville)
- Pour les autres villes : **5 € + 0,59 €/km** (distance calculée automatiquement)
- Le calcul est effectué côté frontend (preview) ET recalculé côté backend (facturation) via le `PricingService` partagé

- Remise de **10 %** automatique si le nombre de convives dépasse le minimum requis de 5 personnes ou plus

8.6 Gestion atomique du stock

La requête SQL de décrémentation utilise une **clause conditionnelle** :

```
UPDATE menus
SET stock = stock - 1
WHERE id = $1 AND stock > 0
RETURNING stock
```

Si `rowCount = 0`, le stock est insuffisant → `ROLLBACK` et erreur 409 (Conflict). Cette approche résout le problème de concurrence sans avoir recours à un verrou pessimiste.

8.7 Refactoring en architecture OOP

Le backend a été restructuré en **architecture en couches** :

```
Route → Controller → Service → Repository → Pool (PostgreSQL)
```

Couche	Responsabilité
Route	Définition des endpoints, middleware (auth, validation)
Controller	Réception HTTP, extraction des paramètres, réponse
Service	Logique métier, règles de gestion, transactions
Repository	Requêtes SQL, accès aux données
Pool	Connexion PostgreSQL (pool de connexions)

Avantages :

- Séparation des responsabilités (principe SRP)
- Testabilité unitaire de chaque couche
- Réutilisation de la logique métier
- Maintenance simplifiée

8.8 Conformité RGPD

Exigence	Implémentation
Consentement (Art. 6)	Checkbox obligatoire à l'inscription
Droit d'accès (Art. 15)	Consultation des données dans l'espace personnel
Droit à la portabilité (Art. 20)	Endpoint `/users/export` — export JSON des données personnelles
Droit à l'effacement (Art. 17)	Endpoint `/users/account` (DELETE) — anonymisation des données, vérification qu'aucune commande active n'existe
Politique de confidentialité	Page légale éditable par l'administrateur

Conclusion

Le projet Vite & Gourmand représente une application web complète couvrant l'ensemble du spectre du développement web moderne : frontend React, backend API REST, base de données relationnelle, authentification sécurisée, déploiement Docker et conformité RGPD.

Les choix techniques (architecture en couches, transactions SQL, JWT, conteneurisation) répondent aux exigences de qualité, de sécurité et de maintenabilité attendues pour un projet professionnel.

La méthodologie Agile simplifiée a permis de livrer le projet de manière itérative, en priorisant les fonctionnalités critiques et en adaptant le périmètre au fil du développement.